Phase 3 Report

March 24, 2026

Project/Course: CMPT 276 D300, Phase 3
Team Name: Group 2
Team Members: Harman Bains, Nathaniel Chen, Jayden Pai, Komal Kaur, Maryna Almeida

1 Unit Testing

Unit testing was used for most classes to confirm that each of their respective methods worked between themselves. For the Shield class, we identified various features that required unit testing. First, we tested the chest when it was initialized, including if it started as uncollected, and if the value, effect duration, spawn status, and the time until it spawns again was all correctly set. For the Coin class, we identified that the key features of the class that must be unit tested were if its value and collection status was set properly once initialized, and if the collect and uncollect methods from reward set these values properly.For the Chest class, we once again identified and tested the initialized version of the chest to ensure all its values were set properly, and tested the feature where it can only open at a specific value of coins, which is set from the chest's constructor. Although we did not test the Reward class directly, we did use all of its methods within each subclass, which allowed us to still test their methods and confirm they work. For the LosingByCoins class, we first tested the constructor to ensure the values are properly initialized. We also tested each feature for grabbing dimensions, such as the starting x and y values for the window and the button inside the window, as well as the width and height of the window and box inside the window. Additionally, we tested the feature of whether a click is inside of a button by passing it various X and Y values. Lastly, we tested whether the background, title and back button correctly draw themselves in the paintComponent method.

Unit testing also started with the cell Java file. This is where it tested the basic methods such as getters and setters being connected data including the position, cell type and entity along with testing the public cell method itself to define. This meant the entity java file also being covered as a result. There was also a testing conducted on removed entity method which does it off a cell would then connect to another method to test called is empty. This was a boolean type that required two tests since it needed to be checked whether the entity was empty or not from removed entity in the cell becoming a double coverage case. Unit testing was also considered in the method is traversable since it required the enum class cell type between the cell Java file which both were tested. This is ensuring a boolean coverage that the hedge or tree ahead can block the explorer or does not appear ahead of path. The next part in unit testing was in the abstract static enemy class which extends entity with also having its subclasses creating an inheritance called thorns and bear traps. The first part was testing to define the method of static enemy and its inherited subclasses underneath with the appropriate parameters. Then it came further basic unit tests in static enemy with the basic get and set methods of game and damage value. Unit testing for the explorer class had parts covered from other classes resulting in only half of the methods remaining to test. The set and get methods of facing were covered at first after defining an explorer in the testing as its regarding where the explorer is directed. The next is updating the shield and get method for shield

ticks remaining as it required looking into multiple different durations for the update shield method which zero is the baseline and checking through shield is active would verify successful tests. The last method to check was the collect reward which needed to test if a it is a type shield for activation, coins to update score, and the last type being neither but a reward that checked collect method was reached.

Unit testing for moving enemies Java files were also done. It covered pixel conversion, movement validation, direction updates and opposite direction logic. For movement tests, the stating position on the map was critical, as calling move() triggers collision detection based on the cell type the enemy would move to into, position (8, 9) was used to test invalid moves, as it would collide to a hedge, while (7, 9) was used for valid movement. The updateDirection method was tested for all four directions, verifying that the correct velocity vectors were set through the velocity getters. The opposite direction logic was tested similar to updateDirection as it tested all four directions. The updateMovement method was the most challenging as it contains do-while loop that randomly picks a new direction while avoiding the opposite of the current one. For the individual enemy subclasses, Pirate and Animal, only two tests were required, verifying their enemy type and confirming that the inherited updateMovement behaviour from MovingEnemy functioned correctly.

For the WinningView class, we tested that the object was successfully created, and that the panel dimensions correctly matched the game map dimensions multiplied by the tile size of 32. We also tested that the width and height values were always positive to ensure no out of bounds values were set. Finally, we tested that the paintComponent method executed without throwing any exceptions using a BufferedImage as a fake graphics context. For the HomePage class, we tested the constructor to ensure all values were properly initialized. We tested each getter for the button dimensions, specifically the width and height, as well as the button position calculations. We tested the isClickInsideButton feature by passing coordinates inside, outside the button, on the exact edge, and each individual boundary condition when x and y were too small or too large. We also tested the handleMouse method with coordinates both inside and outside the button, and tested that paintComponent executed without exceptions. For the LosingbyHearts class, we tested the constructor to ensure all values were properly initialized. We tested each getter for the button dimensions. We tested the isClickInsideButton feature with coordinates inside, outside, on the exact edge, and each individual boundary condition. We also tested the handleMouse method and paintComponent. For the GameView class, we tested the tutorial status getter and setter, the map toggle feature including toggling twice to return to the original state, and the getStartingXCell and getStartingYCell methods to ensure they never returned negative values. We tested stopLoop to ensure it executed without crashing. We tested all drawing methods including three separate branches of paintComponent: the tutorial active branch, the partial map branch with tutorial off, and the whole map branch with map toggled on. All drawing methods were tested using a BufferedImage as a fake graphics context. We also tested specific game states by spawning the shield, collecting the chest, moving the explorer to reward positions, and setting the explorer facing left to cover additional branches in the drawing methods.

Unit testing was also used for the Position and GameMap classes as they are self contained and have minimal external dependencies. The Position class was tested for core functionality including coordinate retrieval, position and hash code equality, euclidean distance calculation, and adjacent position retrieval. Coordinate retrieval tested getters for x and y, using both positive and negative coordinates for position. equals() was tested across several cases: two positions with identical coordinates, comparing a position to null, comparing a position to a non position object, and two positions differing in x and two positions differing in y. hashCode() was also tested to verify that equal positions produce identical hash codes and different positions produce different hash codes to ensure compatibility with hash based collections. distanceTo() was tested with a known right triangle (given position) to verify the Euclidean distance formula in several cases: two calculated distances to the same position should be equal, distance between two positions should be equal in both directions, and calculations involving negative coordinates should function correctly. getAdjacentPositions() was tested to verify that exactly four positions were returned from exactly above, below, right and left of a given position. On the other hand, the GameMap class was tested for core functionality, correct map loading, cell validity, movement validity, and operation on

cells. First, the default constructor was tested to verify that the starting position of the player loads correctly. Since the map loads from a text file, it was tested by verifying the returned height and width against the known dimensions of the text file used. getCell(Position) was tested for correct cell retrieval for positions containing negative x and y, out of bounds x and y, and valid positions. Movement validity was tested for negative x and y, out of bounds x and y, a hedge cell, an empty cell, and a cell containing an entity. These were tested for both the player and moving enemies. In addition, a cell containing the chest (goal) was tested specifically for moving enemies since enemies should not be able to traverse on the chest. removeEntity() was tested by placing an entity on a cell and verifying the cell is empty after removal. Cell types were verified by asserting that the type returned at a given position corresponds to the cell type indicated in the same position on the map text file.

2 Integration Testing

The last part was an integration testing on the static enemy method called on collision which involved the explorer and game Java file. This started with deactivating the shield to operate during test so there could be the interaction between understanding multiple ways how damage can be inflicted on the hearts onto the explorer in the game which is based on subtracting the difference extracted. Resulting in ending up similarly done in bear traps and thorns but being defined under its subclass name as it overrides the method. The integration testing that came into the process was the before each since explorer has parameters of game map which its class needed to load the map and game to set up where the explorer can be positioned alongside interacting within the rewards, static enemies, and moving enemies. This integration testing was also similarly done in explorer inherits being the abstract class of character. It needed a concrete class to define a character in the testing which the move boolean method was testing every direction through the enum direction class the character goes in with the respective enum class along with also testing the null case of no direction. The way assertions were used was to verify with positioning based on the direction applied through move method towards character. The last method was a boolean of can move to which had two methods of a null case of position and map regarding the character method parameters and a out of bounds case regarding negative value coordinates of positioning character.

For the LosingbyHearts class, we tested that when the explorer's hearts reached zero, the game state correctly changed to GameState.LOST, testing the interaction between the Game, Explorer and Losingby-Hearts classes. also, we tested that clicking the back button inside the button boundaries by stopping the GameView game loop, testing the interaction between LosingbyHearts and GameView. For the GameView class, we tested that actionPerformed correctly detected the GameState.LOST state after hearts reached zero, verifying the interaction between Game and GameView. We also tested that when all coins were collected and the explorer reached the chest position, the game state correctly changed to GameState.WON, verifying the interaction between Game, Explorer, Reward and GameView.and that when the tutorial was active, actionPerformed blocked game.tick() from being called, confirming the tutorial correctly paused game updates.

Integration tests were also written for the Game and GameInput classes. GameTest covers the following interactions: first, the initial state was verified by asserting that hearts initialize to 6, max hearts to 6, and score to 0 after construction. startGame() was verified to set the game state to PLAYING and load the map. updateScore() was tested for increments, decrements, and accumulation of points. updateHearts() was tested for valid and correct changes. Setters and getters for State were tested to verify state transitions. checkLoseCondition() was tested for zero hearts, negative score, and the case where neither condition is met. checkWinCondition() was tested for uncollected coins present, all coins collected but explorer not on chest, all coins collected and explorer on chest triggering GameState.WON, and a chest in the rewards list where the explorer is not standing on it. getPixelX() and getPixelY() were tested for both the Explorer and a MovingEnemy to verify both branches of the instanceof check. checkCollision() was tested for full overlap, and no overlap in all four directions, right, left, above, and below to cover all

branches of the boolean expression. game.input() was tested for all four movement keys W, A, S, D, an unrecognized/invalid key, and movement into a hedge/wall. tick() was tested for coin collection, spawned and unspawned shield collection, thorn damage, bear trap damage, shield blocking static enemy damage, invincibility counter incrementing, invincibility resetting after threshold, heart reduction on moving enemy collision, no heart reduction when invincible, no heart reduction when shield is active, no score deduction when already standing on a hazard, chest opening with sufficient coins, chest not opening with insufficient coins, already collected chest not reopening, and lose condition triggered through tick. GameInputTest covers the following interactions between GameInput and GameView: Pressing Q while the tutorial is active deactivates the tutorial. Pressing any other key while the tutorial is active leaves the tutorial active. Pressing M toggles the map on and pressing M again toggles it back off. Pressing a movement key while the map is toggled does not move the explorer, verifying that game.input() is not called when the map is open. keyReleased() and keyTyped() are verified to not affect the game state. Since GameView starts a Swing timer on construction, stopLoop() was called immediately after construction to prevent the game loop from interfering with test assertions.

3 Quality and Line/Branch Coverage

There were multiple measures to ensure quality of test cases were in check. The first was to organize the test code with single line spaces where required. The second is creating names for tests that made sure that there would not be in-line comments required explaining it. The third was to add a concrete class where needed for abstract class test files. The last component to consider was adding a before each strategy applied for tests that required a map and game set up prior to the each unit testing which may also consider integration testing where applicable. Line coverage and branch coverage as one group member was 100 percent on all assigned parts of explorer, static enemies and its subclasses, cell, but the character test. It was a file that was only nearly 100 percent on both coverages because the switch statement default towards returning a false or null direction did not have a NONE option in direction enum which would require changes across many Java files. As a result in the character Java file it has a null case to be checked by a if condition which a test was made for it.

There was 100% line and branch coverage for each applicable reward subclass, those being the Shield, Coin, Chest classes, as well as the Reward class itself. For the LosingByCoins class, we were able to achieve 100% line and branch coverage according to the report. However, after looking further into the report, we noticed that the overridden mouseClicked method inside of the class' constructor misses all four branches. We chose not to create test cases for this, as we were unsure of how to create tests for this method. Additionally, the corresponding methods that use this method are instead given specific X and Y values through the test cases. Additionally, we were able to confirm that, through play-testing our methods during the coding phase, we were able to confirm that this method works at a general level.

The MovingEnemy class achieved 100% line coverage, but fell short of 100% branch coverage in two methods. In updateDirection, the switch statement has no default case since all four directions (UP, DOWN, LEFT, RIGHT) are already handled, however it still flags as a missing branch. In updateMovement, the do-while loop retries when a random direction matches the opposite of the current one, so a test was written that calls updateMovement 100 times in a loop, increasing the likelihood that the retry branch would be triggered by chance. However, since the direction is picked randomly, full branch coverage could not be guaranteed.

For the WinningView class, we achieved 100% line coverage. For the HomePage class, we achieved 100% line coverage and 100% branch coverage for the class itself. For the LosingbyHearts class, we achieved 100% line coverage and 100% branch coverage. For the GameView class, we achieved 94% line coverage and 88% branch coverage overall according to the JaCoCo report. Most methods achieved 100% line and branch coverage including actionPerformed, drawVisibleRewards, drawRewards, drawVisibleBarriers, drawBarriers, drawStaticEntities and drawWholeMap. The remaining uncovered areas were paintRewardHUDs at

65% line and 50% branch coverage, as the shield HUD branch requires the explorer's shield to be active during the paintRewardHUDs call specifically. The drawVisibleMovingEntities method achieved 80% line and 66% branch coverage as the FOX and PIRATE2 enemy type branches required those entities to be within the explorer's visible range simultaneously. The drawMovingEntities method achieved 100% line but 82% branch coverage for similar reasons regarding specific enemy type branches.

The GameInput and Position classes achieved 100% line and branch coverage. The Game class achieved 100% line coverage and 98% branch coverage, while the GameMap class missed 3 lines but had 100% branch coverage. For the Game class, the else if (s instanceof BearTrap) branch in the static enemy damage logic within tick() remained partially covered despite a dedicated test case testTickDeductsScoreWhenExplorerOnBearTrap passing successfully, and producing the expected score deduction of -3. This could not be resolved and is the only partially covered branch in the Game class. For the GameMap class, the line coverage gap corresponds to the catch(FileNotFoundException e) block in loadMap(). This branch is only reachable by passing an invalid file path to loadMap(), which was not tested as all tests use a valid map file in from a hard coded path. A MainTest class was omitted as it was only used to construct UI and run the game, and had no testable logic to assert against. Moreover, all components that Main depends on were already covered by previous tests making further testing redundant.

4 Findings

What was learned about writing and running the tests is how significant it is in breaking down code. This is regarding it not being a simple process, but always continuously modified to maintain the quality of the code is successful. Especially when going through parts such as unit testing ensuring the whole class can operate with other classes with integration testing of the interactions. Overall it is essential to always write tests as it can find the presence of bugs that may not have been captured prior. Changes made to the production code resulted in two common ways found throughout the process. The first was adding a get and set method for a class. It was derived from observing during the testing and warnings given for the class. The second was the a null case not being covered. An example is with a switch statement that had a default case for false which could not be reached, so the alternative was to add a if condition before of a null case. There was noticeable change in the quality of the production code and finding bugs to fix. This can be attributed to assertions which helped solve many incorrect expected and actual number cases. An instance includes the testing in static enemy which needed its on collision method to be corrected to having damage inflict the hearts through the difference and not decrementing.

Through writing and running our tests, we were able to identify various methods, variables and branches that were seemingly unused or skipped during our coding phase. Additionally, while writing and running our tests for the chest class, we identified a variable inside of the chest class that followed the same behaviour as the "collected" boolean. As a result, whenever these two variables were used together, the second branch involving the secondary variable would never run. As a result, we removed the second variable and all associated methods, which allowed us to achieve 100% line and branch coverage for that class. All other reward classes did not have any issues. Furthermore, we were able to reveal a bug in our compilation of JavaDoc comments, that being that we did not add the plugin for JavaDoc comments. By revealing this bug, we were able to improve the quality of our code by now being able to create a JavaDoc index file, which explains each class and method.